# srm_interference icc ブランチの実装内容

干渉判定 3 手法（提案手法 / ΔPER のみ / RSSI のみ）の比較

対象コミット c9cd198795（2026-10-14）

2026 年 10 月 15 日


**この文書の要約**

- 隣接する 2 つの IEEE 802.15.4 PAN（各 30 端末）で、**自 PAN 側の観測だけから相手 PAN の干渉を検知できるか**を評価する。
- icc ブランチは、提案手法（RSSI ビン内 ΔPER 分散の最大値）と、そこから材料を 1 つずつ抜いた 2 つのベースライン（ΔPER **のみ・RSSI のみ**）を**同じデータ・同じ規則**で比べる構成になっている。
- 評価指標は **AUC のみ**、測定モードは **simultaneous のみ**。出力図は相手 PAN 負荷 100% 断面の**折れ線グラフのみ**。
- シミュレーション条件（interference_2pan_config.py・create_csv.py）は save ブランチから変えておらず、変更は解析・作図側だけ。
- ./script/sbatch_jobs.sh 1 本で、config 生成 → 30,000 ランのシミュレーション → トレース集計 → 3 手法の解析 → 作図まで通る。


## 目次


1 **実験の全体像** 3
- 1.1 シナリオ . . . 3
- 1.2 無線・MAC・トラフィック . . . 3
- 1.3 スイープ規模 . . . 4
- 1.4 時間軸と測定モード . . . 4

2 **処理パイプライン** 4

3 **PER と ΔPER の定義** 5

4 **干渉判定の 3 手法** 5
- 4.1 共通の枠組み . . . 5
- 4.2 手法 1：提案手法（proposed） . . . 6
- 4.3 手法 2：ΔPER のみ（dper） . . . 7
- 4.4 手法 3：RSSI のみ（rssi） . . . 7
- 4.5 3 手法の対照 . . . 7

5 評価：AUC 8
5.1 なぜ AUC だけか . . . . . 8
5.2 定義と計算 . . . . . 8
5.3 検知結果 CSV の列 . . . . . 9
5.4 折れ線グラフの読み方 . . . . . 9

6 このブランチで意図的にやっていないこと 9

7 実行方法と注意 9

付録 A ファイル構成 10

# 1　実験の全体像

## 1.1　シナリオ

2 つの PAN（PAN1・PAN2）がそれぞれコーディネータ 1 台と端末 30 台からなり、互いに隣接して動作する。PAN1 のコーディネータは原点、PAN2 のコーディネータは $x$ 軸上の距離 $D$ に置き、端末は各コーディネータを中心とする円内に**面積一様**にランダム配置する（半径方向に $r = R\sqrt{u}$）。

表 1　チャネル定義（`interference_2pan_config.py` の `CHANNELS`）

| ch | 中心周波数 | 伝送速度 | チャネル幅 | 受信感度 | 配置半径 $R$ |
|---|---|---|---|---|---|
| 0 / 3 | 920 / 921 MHz | 50 kbps | 100 kHz | −97.00 dBm | 1400 m |
| 1 / 4 | 920 / 921 MHz | 100 kbps | 200 kHz | −93.99 dBm | 1200 m |
| 2 / 5 | 920 / 921 MHz | 200 kbps | 400 kHz | −90.98 dBm | 1000 m |

- **干渉あり／なしの作り分け**：2 つの PAN の中心周波数が同じ（ch 0–2 どうし）なら「干渉あり（interf）」、片方を 921 MHz 側（ch 3–5）に置けば「干渉なし（no_interf）」。
- **帯域ペア**：干渉あり 6 通り {(0,0),(0,1),(0,2),(1,1),(1,2),(2,2)} と、それぞれに対応する干渉なし 6 通り {(0,3),(0,4),(0,5),(1,4),(1,5),(2,5)} の計 12 パターン。ラベルは `50vs50`, `50vs100`, `50vs200`, `100vs100`, `100vs200`, `200vs200`。
- **PAN 間距離**：$D = \max(R_1, R_2) + 100$ m（例：50vs50 なら 1500 m）。
- **対設計**：配置の乱数は Seed で固定（`np.random.seed(seed)`）し、干渉あり／なしで同じ伝送速度なら $R$ も同じなので、**同じ Seed の interf/no_interf は端末配置が同一で、周波数だけが違う**。

## 1.2　無線・MAC・トラフィック

- 送信電力 13.01 dBm（20 mW）、雑音指数 6 dB、CCA は ED のみ（ED しきい値 = 受信感度 + 10 dB）。
- 伝搬：Rayleigh フェージング（1 km/h）＋ 対数正規シャドウイング（$\sigma = 6.8$ dB）。チャネル間干渉はスペクトルマスクから計算。
- MAC：CSMA/CA（minBE 3, maxBE 5, maxCSMABackoffs 4）、ACK あり、最大再送 3 回。
- フレーム長 250 B。各 PAN でコーディネータ→各端末（DL）30 フロー、各端末→コーディネータ（UL）30 フローの計 60 フロー。到着過程は**ポアソン**。
- **提供負荷**：PAN 全体の提供レートが「衝突なしでデータ＋ ACK を 1 回やり取りする所要時間」から決まる飽和容量の $L\%$ になるよう、1 フローあたりのレートを決める（`offered_load_pps`）。$L \in \{20, 40, 60, 80, 100\}$ を両 PAN で独立に振る（$5 \times 5 = 25$ 通り）。

表 2　負荷換算の目安（1 フローあたり、測定窓 200 s 中のフレーム数）

| 伝送速度 | 1 回の交換時間 | 負荷 20% | 負荷 100% |
|---|---|---|---|
| 50 kbps | 50.3 ms | 約 13 フレーム | 約 66 フレーム |
| 100 kbps | 28.4 ms | 約 24 フレーム | 約 118 フレーム |
| 200 kbps | 17.4 ms | 約 38 フレーム | 約 191 フレーム |

低負荷・低速側では 1 端末あたりのフレーム数が少なく、PER の推定値に二項ノイズ（分散 $p(1-p)/n$）が大きく乗る点が、後述の判定手法の設計に効いてくる。

### 1.3 スイープ規模

12 帯域パターン × 5（PAN1 負荷）× 5（PAN2 負荷）× 100 Seed = **30,000 ラン**。1 ランのシミュレーション時間は 1060 s。

### 1.4 時間軸と測定モード

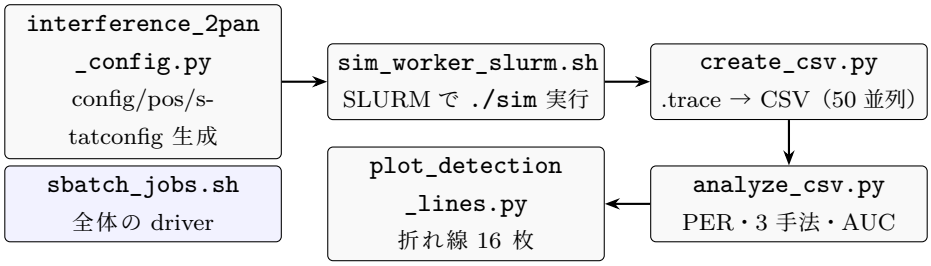
IEEE 802.15.4 の標準では、PER の計算に要る MAC カウンタを同時に測定できない。そのため、トレースには次の時間割で窓が定義されている。

| 時刻 [s] | 内容 |
|---|---|
| 0–20 | ウォームアップ（送信なし） |
| 20–220 | W0：macTxSuccessCount |
| 220–420 | W1：macRetryCount |
| 420–620 | W2：macMultipleRetryCount |
| 620–820 | W3：macTxFailCount |
| 820–1020 | W4：macRxSuccessCount |
| 1020–1060 | ドレイン（生成停止、残りのフレームを捌く） |

- **sequential**：各カウンタを自分の窓で順番に測る（標準に忠実）。
- **simultaneous**：全カウンタを W0（20–220 s）で同時に測る。標準では実現できない「上界（参照）」。

`create_csv.py` は 1 本のトレースから両方を集計するが、**`icc` ブランチの解析は simultaneous だけを読む**（`COLUMN_SUFFIX = "_simultaneous"` 固定）。見たいのは時間分割のコストではなく手法間の差なので、測定側の劣化が無い条件に固定して手法だけを比べる、という方針である。

## 2 処理パイプライン




1. **生成**：Jinja2 テンプレートから全組み合わせ分の `.config`/`.pos`/`.statconfig` を作る。2,000 件ずつのバッチで生成する。
2. **シミュレーション**：1 ラン 1 ジョブで SLURM に投入し、バッチ内の全ジョブ完了を待つ。
3. **集計**：`.trace` を 50 個の SLURM ジョブに分けて解析し、部分 CSV を `plots/simulation_results.csv` にマージする。行数が manifest と一致することを確かめてから追記し、そのバッチの中間ファイル（`.trace` 等）を削除する。
4. **解析**（`run_analysis.sh` → `analyze_csv.py`）：PER と ΔPER を計算し、3 手法それぞれの Seed ごとのスコアと AUC を出す。

5. **作図**（`plot_figures.sh` → `plot_detection_lines.py`）：AUC の折れ線を出す。

表 3　主な出力

| パス | 内容 |
|---|---|
| `plots/simulation_results.csv` | 全ランのトレース集計（約 155 MB） |
| `plots/interference_detection_results_simultaneous.csv` | 検知結果。900 行 = 6 帯域ペア × 25 負荷 × 2PAN × 3 手法 |
| `plots/detection_scores_simultaneous.csv` | Seed ごとの干渉指標。180,000 行 |
| `plots/detection_lines/` | 折れ線 16 枚 + 描画値 CSV |

# 3　PER と ΔPER の定義

端末 $d$ の上り（UL：端末→コーディネータ）と下り（DL：コーディネータ→端末）それぞれについて、W0 で測った値から

$$\mathrm{PER} = 1 - \frac{N_{\mathrm{rx}}}{S + R + M + F}$$

を計算する（`_one_side_per()`）。分子 $N_{\mathrm{rx}}$ は**受信側**が実際に受信したデータフレーム数、分母は**送信側**の MAC カウンタ（macTxSuccessCount $S$、macRetryCount $R$、macMultipleRetryCount $M$、macTxFailCount $F$）の和である。macCsmaFailCount は「混んでいて送れなかった」ものなので分母から外している。値は $[0, 1]$ にクリップする。

そのうえで、端末ごとに

$$\boxed{\Delta\mathrm{PER}_d = \mathrm{PER}_{\mathrm{DL}}(d) - \mathrm{PER}_{\mathrm{UL}}(d)}$$

を作る。相手 PAN の送信は、自 PAN のコーディネータと端末とで受け方が異なる（位置が違う）ため、干渉があると DL と UL の失敗率が**非対称に**崩れる、というのが ΔPER を使う発想である。

**既知の問題（未修正・把握済み）**

分母から macCsmaFailCount を外しているため、再送の途中で CSMA 失敗に分類されたが実際には相手に届いていたフレームがあると、受信数が分母を上回り PER が負になる。simultaneous での実測クリップ率は約 0.28%。3 手法が同じ PER を使うので、手法間の比較には影響しない。

# 4　干渉判定の 3 手法

## 4.1　共通の枠組み

どの手法も、ある条件（帯域ペア・両 PAN の負荷・どちらの PAN から見るか）について、**Seed ごとに 1 つのスカラー（干渉指標）**を出す。値の向きは 3 手法とも「**大きいほど干渉あり**」に揃えている（`METHOD_SIGN` がすべて $+1$）。

Seed $s$ における自 PAN の端末集合を、**1 フレームでも受信できた端末**に絞る：

$$\mathcal{D}_s = \{\, d \;:\; \overline{\mathrm{RSSI}}_d \neq 0 \,\}$$

ここで $\overline{\mathrm{RSSI}}_d$ は「コーディネータが端末 $d$ から受信した UL データフレームの RSSI の平均（W0 内）」で、受信が 0 件のときは 0.0 が入る（これを番兵に使う）。この絞り込みは `_iter_seeds()` 1 箇所に集約

されており、**3 手法が同じ端末集合を見る。**

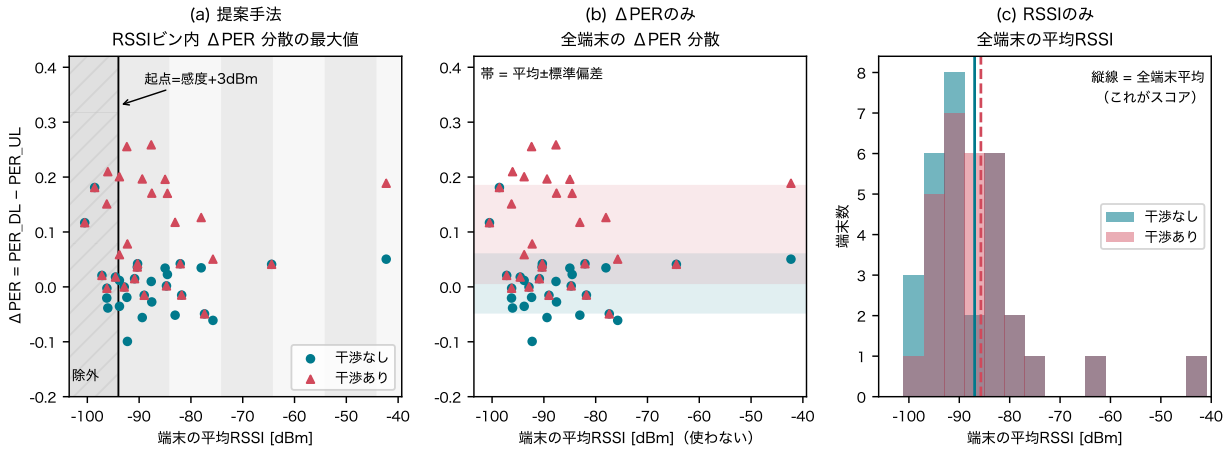



図 1　3 手法が何を見ているかの模式図（説明用の合成データであり、シミュレーション結果ではない）。(a) 提案手法は RSSI でビン分けしてビン内で ΔPER のばらつきを見る。(b) ΔPER のみは全端末をまとめてばらつきを見る。(c) RSSI のみは ΔPER を使わず平均 RSSI だけを見る。

## 4.2　手法 1：提案手法（proposed）

**RSSI ビンごとの** ΔPER **分散の、Seed 内での最大値**。ΔPER と RSSI の両方を使う。

1. ビンの起点 $a_{\mathrm{ch}} =$ (受信感度) + 3 dBm をチャネルごとの固定値として置く（50 kbps: −94.0 dBm、100 kbps: −91.0 dBm、200 kbps: −88.0 dBm）。
2. $a_{\mathrm{ch}}$ から上へ幅 10 dBm のビン $B_k = (a_{\mathrm{ch}} + 10k,\ a_{\mathrm{ch}} + 10(k+1)]$ を、最大 RSSI を覆うまで作る（`select_bins()`）。
3. 端末が 2 台以上入るビンだけを使う。$a_{\mathrm{ch}}$ 以下の端末（セル端）はどのビンにも入らない。
4. 各ビンで ΔPER の分散（母分散、`np.var`）を取り、その最大値をスコアとする：

$$\mathrm{score}_{\mathrm{proposed}}(s) = \max_{k:\,|B_k \cap \mathcal{D}_s| \geq 2} \mathrm{Var}\big(\{\Delta\mathrm{PER}_d : d \in B_k \cap \mathcal{D}_s\}\big)$$

**■考え方**　干渉が無ければ、RSSI が同程度（＝コーディネータからの距離・リンク品質が同程度）の端末どうしは ΔPER も似た値になるはずである。相手 PAN の干渉は端末の位置によって効き方が違うので、**同じリング内の端末どうしで** ΔPER **がばらつく**。RSSI で揃えることで、「距離によるリンク品質の差」から来るばらつきを取り除き、干渉によるばらつきだけを見ようとしている。

**■起点を「受信感度 + 3 dBm」に固定している理由**

- セル端の端末は PER が 0.5 付近になり、二項ノイズ $p(1-p)/n$ が最大になる。干渉が無くても端末間のばらつきが大きく出るので、そこを落とさないと誤検知の床が上がる。
- 一方で落としすぎると、ビン内の端末数が減って検出力が下がる。伝搬モデルでの概算では、+3 dBm で約 22 台が残り最大ビンの端末数は約 15 台（ED しきい値 = +10 dB を起点にすると約 10 台しか残らない）。
- 以前は「その Seed の実測 RSSI 最小値 + 5 dBm」を起点にしていたが、データ依存なのでビン境界が負荷や帯域で揺れていた。固定値にしたことで、ビン境界は Seed にも負荷にも依存しない。
- 干渉ありとなしの対（例：ch0 と ch3）は受信感度が同じなので、起点も同じになる（コードで検査

している）。

### 4.3 手法 2：ΔPER のみ（dper）

**全端末の ΔPER の分散**。ビン分けしない。

$$\mathrm{score}_{\mathrm{dper}}(s) = \mathrm{Var}\big(\{\Delta\mathrm{PER}_d : d \in \mathcal{D}_s\}\big)$$

RSSI は「受信できたか」（$\mathcal{D}_s$ への絞り込み）にしか使わない。端末が 2 台未満ならスコアを作れない。

**■位置づけ**　提案手法から `select_bins()` を外したもの。提案手法との差が「RSSI で揃えたことの寄与」を表す。

**読み方の注意：差に含まれるものは 2 つ**

`select_bins()` は「ビンに分ける」ことに加えて「起点 $a_{\mathrm{ch}}$ 以下のセル端端末を落とす」ことも担っている。`dper` はこの両方を外しているので、提案手法との差は**「ビン分割の効果」と「セル端除外の効果」の合算**になる。どちらが効いているのかを分けたい場合は、「セル端だけ落として全端末の分散を取る」中間の手法が別途必要になる（現在の実装には無い）。

### 4.4 手法 3：RSSI のみ（rssi）

**全端末の平均 RSSI**。ΔPER を一切使わない。

$$\mathrm{score}_{\mathrm{rssi}}(s) = \frac{1}{|\mathcal{D}_s|} \sum_{d \in \mathcal{D}_s} \overline{\mathrm{RSSI}}_d \quad [\mathrm{dBm}]$$

**■想定している仕組み**　干渉があると、弱い（フェージングで落ち込んだ）フレームから先に失われる。受信できたフレームだけで RSSI を平均するので、平均値が**上振れする**はず、という想定で「大きいほど干渉あり」（+1）の向きに置いている。

**■注意点**

- 向きは自動では決めていない。想定が外れていれば、AUC が 0.5 を下回るという形で結果に現れる（§5）。向きを変えたいときは `METHOD_SIGN["rssi"]` を $-1$ にする。
- RSSI はコーディネータが受けた UL フレームのものだけで、DL 側の情報は入らない。
- 平均は「端末ごとの平均 RSSI」を端末について等重みで平均したもの（フレーム数で重み付けしていない）。

### 4.5 3 手法の対照

**■ベースラインの設計意図**　2 つのベースラインは、提案手法から**材料を 1 つずつ抜いたもの**になっている（アブレーション）。

- `proposed` > `dper` なら、RSSI でそろえる（＋セル端を落とす）ことが効いている。
- `proposed` > `rssi` なら、ΔPER を見ることが効いている（RSSI の変化だけでは干渉を捉えられない）。

表 4　3 手法の比較

| | 提案手法（proposed） | ΔPER のみ（dper） | RSSI のみ（rssi） |
|---|---|---|---|
| 使う量 | ΔPER + RSSI | ΔPER | RSSI |
| 集約 | ビン内分散の最大値 | 全端末の分散 | 全端末の平均 |
| セル端の除外 | あり（感度 +3 dBm 以下） | なし | なし |
| 単位・値域 | 無次元、$[0, 1]$ | 無次元、$[0, 1]$ | dBm（実測 $-60$ 前後） |
| 縮退条件 | 有効ビンが 0 個 | 端末 2 台未満 | 端末 0 台 |
| 縮退時の値 | 0.0 | 0.0 | $-\infty$ |
| 実装 | `_score_proposed` | `_score_dper` | `_score_rssi` |

**■縮退（スコアを作れなかった Seed）の扱い**　スコアを作れない Seed には「干渉の証拠が最も弱い」側の端の値を入れる。`rssi` だけ $-\infty$ なのは、dBm に下限が無く、0.0 を入れると実測値（$-60$ dBm 前後）より遥かに大きく「最強の陽性証拠」に化けてしまうためである。縮退した件数は検知結果 CSV の `n_degenerate_*` 列に残る（`save` の実データでは提案手法の縮退は 0 件）。

**■実装上の配慮**　ΔPER と RSSI の配列は条件ごとに 1 度だけ作り、3 手法で共有する（155 MB の CSV を 3 回読まない）。ビン構成の診断値（ビン内端末数、縮退 Seed 数）は `proposed` のときだけ集計し、同じ Seed を 3 回数えないようにしている。合成データで、リファクタ後の `proposed` の AUC が `save` と完全に一致することを確認済み。

# 5　評価：AUC

## 5.1　なぜ AUC だけか

`save` では F1 も出していたが、F1 は閾値を 1 点決める必要がある。3 手法のスコアは分散（無次元）と平均 RSSI（dBm）で単位も値域も違うので、**共通の閾値グリッドが引けない**。AUC は順位しか見ないため単調変換に不変で、単位の違う指標を同じ土俵で比べられる。

## 5.2　定義と計算

各条件・各手法について、干渉あり 100 Seed のスコア（陽性）と干渉なし 100 Seed のスコア（陰性）を作り、

$$\mathrm{AUC} = P(\mathrm{score}_{\mathrm{interf}} > \mathrm{score}_{\mathrm{no_interf}}) + \tfrac{1}{2}P(\text{同値})$$

を順位（Mann–Whitney U）で計算する（`_rank_auc()`）：

$$\mathrm{AUC} = \frac{R_{\mathrm{pos}} - n_{\mathrm{pos}}(n_{\mathrm{pos}} + 1)/2}{n_{\mathrm{pos}}\, n_{\mathrm{neg}}}$$

$R_{\mathrm{pos}}$ は 2 群を連結して振った通し順位（同値は平均順位）のうち陽性側の和。1.0 で完全分離、0.5 でランダムと同等、0.0 で完全に逆転。CSV には小数 4 桁で書く（$100 \times 100$ ペアなので刻みは $10^{-4}$）。

**向きは補正しない**（$\max(\mathrm{AUC}, 1 - \mathrm{AUC})$ にしない）。0.5 を下回ったら「指標が干渉の有無と逆相関している」という結果そのものであり、補正するとベースラインが実際より強く見えてしまう。公平さは「3 手法とも同じ規則で扱う」ことで担保する。

### 5.3 検知結果 CSV の列

```
bandwidth, distance, pan1_offload, pan2_offload, pan, method, auc,
n_interf_seeds, n_no_interf_seeds, n_degenerate_interf, n_degenerate_no_interf
```

### 5.4 折れ線グラフの読み方

- **相手 PAN 負荷 = 100% の断面**だけを描く。「相手が最大負荷のとき、自分の負荷を上げると検知性能がどう落ちるか」を見るため。
- 横軸：自 PAN 負荷 20–100%、縦軸：AUC。
- 色：相手 PAN の帯域（50 / 100 / 200 kbps）。線種：判定手法（提案手法＝実線、ΔPER のみ＝破線、RSSI のみ＝点線）。
- 対称ペア（50vs50 など）は PAN1 側だけを採り、全線を 100+100 Seed に揃える。
- 図は手法ごと 3 種＋ 3 手法重ね描き（`compare`）1 種、それぞれ自帯域 3 枚＋ 3 パネル 1 枚で計 16 枚。
- y 軸の下端は通常 0.5 だが、ベースラインが 0.5 を割る値を含むときは全図共通で 0.1 刻みに下げる（図をまたいで比較できるよう、枚ごとには変えない）。

**■参考：`save` での提案手法の値**　simultaneous での AUC 中央値は 0.966。自 PAN 負荷別では 20%: 0.998 → 100%: 0.887 と、自 PAN の負荷が上がるほど自 PAN 内の輻輳由来のばらつきが干渉の痕跡を覆い、性能が落ちる。3 手法の比較結果は `icc` で回した後に得られる。

## 6 このブランチで意図的にやっていないこと

- F1 スコア（共通の閾値グリッドが引けないため）。`save` に実装あり。
- ヒートマップ（負荷 $5 \times 5$）と ROC 曲線。手法を重ねられるのが折れ線だけのため。`save` に実装あり。
- sequential モードの解析。
- 変動係数・正規化分散による判定（`cv` ブランチ）。
- PER のクリップの修正（3 手法に等しく効くため比較には影響しない）。
- `base_simulator` の変更。

## 7 実行方法と注意

| 目的 | コマンド |
|---|---|
| シミュレーションから図まで全部 | `./script/sbatch_jobs.sh` |
| 既存 CSV から解析と作図だけ | `./script/sbatch_reanalyze.sh` |
| 既存の検知結果 CSV から図だけ | `./script/plot_figures.sh` |

**実行前の確認**

- 計算機側が icc ブランチの c9cd198795 になっていること（git status -sb と git log --oneline -1）。
- sbatch_jobs.sh は起動直後に既存の plots/simulation_results.csv を削除する。残したい CSV は事前に退避する。
- シミュレーション条件は save と同じなので、同条件の simulation_results.csv（_simultaneous 列を含む）が既にあれば、sbatch_reanalyze.sh だけで 3 手法の結果が出る。
- AUC の計算に scipy が必要。

# 付録 A　ファイル構成

| ファイル | 役割 |
|---|---|
| script/interference_2pan_config.py | config/pos/statconfig の生成。チャネル・窓・負荷換算の定義元 |
| script/sim_worker_slurm.sh | 1 ランを走らせる SLURM ワーカー |
| script/create_csv.py | .trace → simulation_results.csv（sequential/simultaneous 同時集計） |
| script/analyze_csv.py | PER・3 手法のスコア・AUC・検知結果 CSV |
| script/plot_detection_lines.py | 折れ線グラフ |
| script/run_analysis.sh | 解析から作図まで（下の 2 つから呼ばれる） |
| script/plot_figures.sh | 図だけ作り直す |
| script/sbatch_jobs.sh | 全体の driver |
| script/sbatch_reanalyze.sh | 解析と作図だけやり直す |
| template/*.j2 | Jinja2 テンプレート |
| docs/README.md, docs/auc.md | ブランチの設計文書 |